

FPGA-Based MPCC for the Ultra96-V2: Design Considerations

This document outlines considerations for implementing Model Predictive Contouring Control (MPCC) on the Ultra96-V2 FPGA, based on our experience with Pure Pursuit and an analysis of what a more advanced controller would require.

Contents

1	Starting Point: What We Know from Pure Pursuit	3
2	Why Move Beyond Pure Pursuit	3
3	The Algorithm at a High Level	3
4	Solver Selection: Why ADMM Looks Promising for FPGA	4
5	FPGA Performance Estimates	5
5.1	Problem Dimensions	5
5.2	Parallelism in the QP Solver	5
5.3	Overall Timing	5
5.4	Resource Usage	5
6	Vehicle Model Considerations	6
7	Track Boundary Constraints	7
8	Dynamic Obstacle Handling	7
8.1	Planner-Based Avoidance	7
8.2	Obstacle Constraints in the MPCC	8
8.3	Recommendation	8
9	System Architecture: Jetson ↔ Ultra96	8
9.1	Per-Cycle Communication	8
9.2	AXI-Lite Register Interface	9
9.3	Fallback Controller	10
10	Achievable Control Frequencies	10
11	Development Path	10
12	Summary of Key Design Decisions	11

13 References and Further Reading	11
13.1 MPCC for Autonomous Racing	12
13.2 Real-Time Iteration (RTI) and Embedded NMPC	12
13.3 ADMM and QP Solvers for Embedded Systems	12
13.4 FPGA-Accelerated MPC	12
13.5 Vehicle Dynamics and Tire Modeling	13
13.6 F1TENTH Platform	13
13.7 HLS and Xilinx Tools	13

1 Starting Point: What We Know from Pure Pursuit

The Pure Pursuit controller was our first complete FPGA implementation and served as a proof of concept for the entire workflow. It runs on the Zynq UltraScale+ ZU3EG (part `xczu3eg-sbva484-1-i`) at 100 MHz, consumes minimal resources, and completes a computation in 92–146 clock cycles (roughly 1–1.5 μ s).

On the real Ultra96 hardware, end-to-end compute time (including register writes, start, and readback) measures around 2.3–3.1 μ s. The ROS 2 overhead surrounding each call adds another 1–2 ms, which is mostly serialization and transport—not the FPGA’s concern.

Resource-wise, Pure Pursuit uses about 8 BRAM tiles (4%), 74 DSP slices (21%), 18,700 LUTs (27%), and 7,400 flip-flops (5%). This leaves plenty of room on the FPGA for a more complex controller.

Key takeaways from this first implementation:

- Vitis HLS 2025.1 requires `vitis-run -mode hls -tcl` (the old `vitis_hls` binary is deprecated and co-simulation is broken through it).
- Scalar AXI-Lite interfaces are more reliable for co-simulation than struct-based DMA approaches.
- Q16.16 fixed-point arithmetic maps cleanly to DSP slices and avoids the need for floating-point IP.
- The development cycle ($C \rightarrow \text{synthesis} \rightarrow \text{co-sim} \rightarrow \text{bitstream} \rightarrow \text{deploy}$) takes roughly a day once the workflow is established.

2 Why Move Beyond Pure Pursuit

Pure Pursuit is a geometric controller. It follows a pre-computed raceline by steering toward a look-ahead point, but it has no notion of speed optimization, constraints, or prediction. The car simply follows the path at whatever speed is externally commanded.

For competitive autonomous racing, this has clear limitations. The car cannot reason about upcoming corners, cannot adjust its speed optimally, and cannot react to obstacles through its control law. It also cannot respect physical limits like the friction circle—it just does what it is told and hopes for the best.

Model Predictive Control addresses these issues by solving an optimization problem at every control step: given where the car is now, what sequence of controls over the next N time steps minimizes a cost function while respecting constraints?

MPCC goes one step further by reformulating the tracking problem. Instead of tracking a reference speed profile, MPCC maximizes progress along the path while staying within physical limits. This means the controller naturally finds the fastest safe speed for any given path geometry—no pre-computed speed profile needed.

3 The Algorithm at a High Level

An MPCC controller, in simplified terms, does the following each control cycle:

1. **Receive the current vehicle state** (position, heading, velocity, yaw rate, etc.) from localization.
2. **Receive N reference points** describing the local path to follow, including track boundary widths.
3. **Linearize the vehicle model** at each of the N prediction points (or at the current state, depending on the formulation).
4. **Build a quadratic program (QP)** whose solution gives the optimal control sequence.
5. **Solve the QP** using an iterative method.
6. **Apply the first control** from the optimal sequence and repeat.

For real-time applications, a scheme called Real-Time Iteration (RTI) is often used: instead of solving the nonlinear program to full convergence, only one or a few iterations of a Sequential Quadratic Programming (SQP) method are performed per control cycle. Since the controller runs at high frequency, the solution is continually refined and stays close to optimal.

4 Solver Selection: Why ADMM Looks Promising for FPGA

Our current MPC implementation uses Projected Gradient Descent with per-variable Gershgorin step sizes. This works well on CPU and converges quickly (around 5 iterations typically, up to 20 in difficult cases). However, it is not ideal for FPGA because each iteration involves a dense matrix-vector multiply followed by an element-wise projection—and the two operations cannot easily overlap.

The Alternating Direction Method of Multipliers (ADMM) is an alternative that may map better to hardware:

- Each iteration involves a fixed matrix-vector multiply (using a pre-factored matrix that does not change within a solve), followed by element-wise soft-thresholding and projection.
- The matrix-vector multiply is the bottleneck and is highly parallelizable—exactly the kind of operation where FPGAs tend to excel.
- ADMM typically converges in 10–30 iterations for warm-started problems, which is competitive with our current solver.

The key insight is that the matrix $K = (H + \rho I)^{-1}$ can be factored once at the start of each solve and then reused for all iterations. On FPGA, this factored matrix can be stored in partitioned BRAM, allowing parallel access across rows.

There are other solvers worth considering—for example, OSQP (which also uses ADMM internally) and active-set methods. The choice depends partly on how well the solver’s inner loop maps to the available DSP and memory architecture. For the ZU3EG with its 360 DSP slices, ADMM appears to be a reasonable fit, but this is not necessarily the only viable approach.

5 FPGA Performance Estimates

These estimates are rough and based on counting operations in the algorithm. Actual synthesis results would likely differ, but they give a sense of what might be feasible.

5.1 Problem Dimensions

For MPCC with $N = 40$ prediction steps and 3 control variables per step (steering angle, longitudinal force, and path progress variable):

- Decision variables: $40 \times 3 = 120$
- Hessian matrix: $120 \times 120 = 14,400$ entries
- Constraints: ~ 320 (box constraints on controls + track boundary constraints)

5.2 Parallelism in the QP Solver

The matrix-vector multiply $K \cdot z$ (120×120 matrix times 120-vector) dominates compute.

On a CPU (ARM Cortex-A53 at 1.2 GHz), this requires approximately 14,400 multiply-accumulate operations executed sequentially—roughly 14–43 μs depending on cache behavior and instruction scheduling.

On the FPGA, if we allocate ~ 120 DSP slices to compute all rows in parallel, the same operation completes in about 120 clock cycles at 100 MHz—approximately 1.2 μs . This suggests a potential 12–36 $\times$ speedup for this specific operation.

5.3 Overall Timing

Component	ARM A53 (est.)	FPGA (est.)
Model evaluation (40 steps)	$\sim 50 \mu\text{s}$	$\sim 5 \mu\text{s}$
QP construction	$\sim 100 \mu\text{s}$	$\sim 100 \mu\text{s}$
QP solve (20 ADMM iterations)	$\sim 600 \mu\text{s}$	$\sim 48 \mu\text{s}$
Total per control cycle	$\sim 750 \mu\text{s}$	$\sim 150 \mu\text{s}$

Table 1: Estimated compute time comparison per MPCC control cycle.

These numbers are estimates. The QP construction phase is harder to parallelize (it involves building the Hessian from Jacobians, which has sequential dependencies), so it does not benefit as much from the FPGA architecture.

5.4 Resource Usage

Including the existing Pure Pursuit (which could remain as a fallback controller):

Resource	Pure Pursuit	MPCC (est.)	Combined	Available	Usage
BRAM18K	8	30–50	38–58	216	18–27%
DSP48E2	74	120–200	194–274	360	54–76%
LUT	18,700	25,000–35,000	43,700–53,700	70,560	62–76%
FF	7,420	10,000–15,000	17,420–22,420	141,120	12–16%

Table 2: Estimated FPGA resource utilization for MPCC alongside Pure Pursuit.

The utilization would be in the 55–76% range for most resources. This is within typical design margins, though timing closure may require some effort at the higher end of these estimates. Designs above $\sim 75\%$ LUT utilization on UltraScale+ parts can sometimes be challenging to route—but this is a concern to be addressed during implementation, not a fundamental blocker.

6 Vehicle Model Considerations

Our current MPC uses a kinematic bicycle model—four states (x, y, ψ, v) with velocity treated as a direct control input. This is adequate at low speeds where tire slip is negligible, but breaks down when pushing the car toward its friction limits.

For competitive racing, a dynamic model is more appropriate. A commonly used option is the dynamic single-track (bicycle) model with six states:

- Position (x, y)
- Heading ψ
- Longitudinal and lateral velocities (v_x, v_y)
- Yaw rate $\dot{\psi}$

This model captures tire slip and the coupling between longitudinal and lateral forces. The tire forces can be modeled using a simplified Pacejka (Magic Formula) model or a linear tire model with saturation, depending on how much complexity is acceptable.

For FPGA implementation, the key concern with a dynamic model is the number of trigonometric and division operations per evaluation. Each model step requires $\sin$, $\cos$, $\tan$, atan2 , and several divisions. In our Q16.16 fixed-point framework, each trig function takes roughly 50–100 cycles using Taylor series approximation. With 40 prediction steps evaluated in parallel, this is manageable—the trig functions run concurrently across all evaluation units.

The linearization (computing Jacobian matrices $\frac{\partial f}{\partial x}$ and $\frac{\partial f}{\partial u}$) is more involved with a 6-state model but follows the same structure as our current 4-state linearization: compute partial derivatives analytically and evaluate them at the operating point. The Jacobian for a 6-state, 3-control model has $6 \times 6 + 6 \times 3 = 54$ entries per prediction step, versus $4 \times 4 + 4 \times 2 = 24$ for the kinematic model.

Whether a dynamic model is necessary depends on how fast the car actually drives. Below about 3–4 m/s, the kinematic model is adequate. In our simulation environment the car reaches up to 20 m/s, which is well into the regime where tire slip dominates and a dynamic model becomes essential for accurate prediction.

7 Track Boundary Constraints

The current MPC implementation has no constraints preventing the car from leaving the track. This is fine for initial testing but needs to be addressed for real competition.

In an MPCC formulation, the most natural approach is to work in a path-relative (Frenet) coordinate frame:

- The contouring error e_c measures lateral deviation from the reference path.
- Track boundaries become simple bounds on e_c : the car must stay within

$$-w_{\text{left}}(\theta) \leq e_c \leq w_{\text{right}}(\theta)$$

Here, w_{left} and w_{right} are the distances from the reference path to the left and right track edges, which vary along the track. These values can be extracted from the occupancy grid map that the Jetson already uses for localization.

From the FPGA’s perspective, track boundary constraints are straightforward—they add two bound constraints per prediction step (left wall, right wall), bringing the total constraint count from roughly 240 to 320 for $N = 40$. Since these are still simple box constraints in the Frenet frame, the projection step in the ADMM solver does not become significantly more expensive.

The track width data (one w_{left} and one w_{right} per reference point) can be included in the N reference points sent to the FPGA each cycle. This adds $2 \times 40 = 80$ values to the per-cycle data transfer—negligible overhead.

8 Dynamic Obstacle Handling

For head-to-head racing, the car needs to avoid other vehicles on the track. There are several approaches, and the choice affects the overall system architecture more than the FPGA design specifically.

8.1 Planner-Based Avoidance

In this approach, obstacle avoidance is handled by a path planner running on the Jetson (which has access to LiDAR data). The planner generates a collision-free local reference path, and the FPGA’s MPCC simply tracks this path.

The workflow each cycle:

1. The Jetson detects obstacles from LiDAR.
2. A local planner (e.g., gap-follow, RRT*, or lattice planner) modifies the next N reference points to avoid obstacles—essentially shifting waypoints laterally within the track boundaries.
3. The modified reference points are sent to the Ultra96.
4. The FPGA MPCC computes optimal controls to follow the modified reference.

Because the MPCC formulation does not use a reference speed (it maximizes progress along the path subject to physics constraints), laterally shifted waypoints do not cause speed inconsistencies. The MPCC automatically recomputes the optimal speed for the modified path geometry. In tighter corners created by obstacle avoidance, it naturally slows down; on wider paths, it speeds up.

This approach keeps the FPGA design clean—the MPCC does not need to know about obstacles at all. The downside is that the planning and control are somewhat decoupled: the planner might generate a reference that the MPCC then discovers is dynamically infeasible, or the reference might not be globally optimal when considering the car’s dynamics.

8.2 Obstacle Constraints in the MPCC

Alternatively, obstacle avoidance can be incorporated directly into the MPCC’s constraints. For each detected obstacle, a linearized collision avoidance constraint is added at each prediction step:

$$\vec{n}_{j,k}^T \cdot (p_k - p_{\text{obs},j}(k)) \geq d_{\text{safe}}$$

where $\vec{n}_{j,k}$ is the direction from the obstacle to the ego vehicle, and d_{safe} is a safety distance.

This adds 2 constraints per obstacle per time step (approximating the obstacle as a two-sided constraint). For 3 obstacles over $N = 40$ steps, that is 240 additional constraints, bringing the total to roughly 560.

The ADMM solver handles additional constraints without fundamental difficulty—each constraint just adds one more projection operation per iteration. However, the larger constraint set increases memory requirements and may slightly increase convergence time.

This approach gives better results because the controller reasons about obstacles and dynamics simultaneously. It can find maneuvers (like late braking followed by a sharp lane change) that a planner-then-controller pipeline might miss.

8.3 Recommendation

For an initial implementation, the planner-based approach is likely more practical. It separates concerns, keeps the FPGA design focused on control, and does not require the FPGA to handle variable numbers of obstacle constraints. Adding obstacle constraints inside the MPCC is a natural extension if time permits.

9 System Architecture: Jetson ↔ Ultra96

The overall data flow builds on the existing architecture with some modifications for MPCC.

9.1 Per-Cycle Communication

Each control cycle, the Jetson sends:

- Vehicle state: 6–7 values (position, heading, velocities, yaw rate)

- $N = 40$ reference points $\times (x, y, \psi, w_{\text{left}}, w_{\text{right}}) = 200$ values
- Total: ~ 210 values $\times 4$ bytes = ~ 840 bytes

The Ultra96 returns:

- Optimal steering angle, longitudinal force command
- Optional: predicted trajectory for visualization
- Total: $\sim 10\text{--}20$ values = ~ 80 bytes

For communication options:

- **ROS 2 (current)**: Adds 1–2 ms of overhead per round-trip, which is significant compared to the ~ 150 μs FPGA compute time. This overhead comes from serialization, DDS discovery, and middleware processing.
- **Raw UDP**: Estimated 0.3–0.5 ms round-trip over Ethernet. This is roughly $4\text{--}6\times$ less overhead than ROS 2 and leaves more of the time budget available for compute.

The communication path does not need to be all-or-nothing: ROS 2 can be used locally on the Jetson for inter-node communication (LiDAR $\rightarrow$ AMCL $\rightarrow$ planner), while the Jetson-to-Ultra96 link uses raw UDP for minimal latency. The Ultra96 ARM PS receives the UDP packet, writes registers to the FPGA via AXI-Lite, waits for completion, and sends the result back.

9.2 AXI-Lite Register Interface

The current Pure Pursuit design uses a single AXI-Lite control bundle with ~ 29 registers. For MPCC, the register count increases to roughly:

- Control registers (mode, start, status): ~ 5
- Vehicle state: ~ 7
- $N \times 5$ reference values: 200
- Configuration parameters: ~ 15
- Output registers: ~ 5
- Total: ~ 232 registers

AXI-Lite writes take approximately 1 clock cycle each at 100 MHz. Writing 232 registers takes ~ 2.3 μs —fast enough to be negligible. Reading the result (a few registers) is similarly fast.

An alternative is to use AXI DMA or AXI Stream for bulk data transfer, which would be faster for the reference points. However, AXI-Lite is simpler, well-proven in our workflow, and fast enough for the data volumes involved.

9.3 Fallback Controller

The Pure Pursuit controller can coexist on the same FPGA as the MPCC. If the MPCC does not produce a result within the time budget (for example, if the QP solver does not converge), the system can fall back to Pure Pursuit using the pre-loaded raceline. This provides a hardware safety net that requires no software intervention.

10 Achievable Control Frequencies

One of the clearer advantages of the FPGA is enabling higher control frequencies than what is reliably achievable on a CPU.

Control Rate	Budget	FPGA	ARM A53
200 Hz	5.0 ms	Feasible (97% margin)	Feasible (with jitter risk)
500 Hz	2.0 ms	Feasible (92% margin)	Marginal
1 kHz	1.0 ms	Feasible (85% margin)	Unlikely reliable
2 kHz	0.5 ms	Feasible (70% margin)	Not feasible

Table 3: Achievable control frequencies: FPGA vs. ARM Cortex-A53.

The practical benefit of higher control frequency is tighter tracking at the limits of adhesion. At 20 m/s, each control step at 200 Hz covers 100 mm of travel; at 1 kHz, it covers 20 mm. When operating near the friction limit, finer corrections can maintain stability where coarser ones cannot.

Whether this translates into measurably faster lap times depends on many factors—track layout, tire characteristics, localization accuracy—but the capability is there if needed, and it is something a CPU-based implementation would struggle to match reliably.

The FPGA also offers deterministic latency: the compute time is bounded by the hardware design, with no operating system jitter, cache misses, or interrupt handling delays. For a safety-critical real-time system, this predictability can be as valuable as raw speed.

11 Development Path

A reasonable order of implementation, with each phase producing a complete, testable system:

Phase 1: MPC on FPGA (current algorithm, proven workflow)

Take the existing linear MPC with kinematic model and port it to HLS. This is the lowest-risk step since the algorithm is already in C with Q16.16 fixed-point. The main work is replacing `memset/memcpy` with explicit loops, bounding `while` loops in angle normalization, fixing array dimensions, and adding HLS pragmas. The Projected Gradient solver should work initially—switching to ADMM can come later.

Phase 2: Upgrade to dynamic model

Replace the kinematic bicycle model with a dynamic single-track model. This affects

the model evaluation and linearization functions. Test first on CPU (existing test framework), then synthesize for FPGA.

Phase 3: MPCC cost formulation

Change the cost function from position/heading tracking to contouring/lag/progress. Add the progress variable as an additional control. This changes the QP dimensions and cost weights but does not fundamentally change the solver.

Phase 4: Track boundary constraints

Add Frenet-frame track width constraints. This modifies QP constraint construction and requires track width data in the reference points.

Phase 5: ADMM solver

Replace Projected Gradient with ADMM. This is where the major FPGA performance gains come from (parallelized matrix-vector multiply). Test extensively in co-simulation.

Phase 6: Obstacle avoidance

Either implement the planner-based approach (CPU-side, no FPGA changes) or add linearized obstacle constraints to the MPCC.

Each phase is independently testable and produces a working controller. If time runs out at any phase, the system is still functional with whatever has been completed.

12 Summary of Key Design Decisions

Decision	Recommendation	Rationale
Solver algorithm	ADMM (after initial PGD)	FPGA-friendly parallel structure
RTI scheme	1–3 SQP iterations/cycle	Bounded time, high-freq. refinement
Vehicle model	Dynamic single-track	Needed for competitive cornering
Reference data	N points per cycle	Supports dynamic replanning
Track constraints	Frenet-frame bounds	Simple, effective, low overhead
Obstacle avoidance	Planner-based initially	Separation of concerns
Communication	Raw UDP (Jetson–Ultra96)	Minimizes latency
Fallback	Pure Pursuit on same FPGA	Hardware safety net
Fixed-point format	Q16.16 (existing)	Proven, maps to DSPs
Target clock	100 MHz	Achievable on ZU3EG

Table 4: Summary of key design decisions and their rationale.

This document reflects analysis and estimates based on our Pure Pursuit implementation experience and the structure of the existing MPC codebase. Actual synthesis results, convergence behavior, and real-world performance may differ from these projections.

13 References and Further Reading

The following are relevant papers and resources, grouped by topic. These can serve as starting points for deeper investigation and as references in the thesis.

13.1 MPCC for Autonomous Racing

- [1] A. Liniger, A. Domahidi, and M. Morari, “Optimization-based autonomous racing of 1:43 scale RC cars,” *Optimal Control Applications and Methods*, vol. 36, no. 5, pp. 628–647, 2015.
- [2] A. Liniger and J. Lygeros, “A Noncooperative Game Approach to Autonomous Racing,” *IEEE Transactions on Control Systems Technology*, vol. 28, no. 3, pp. 884–897, 2020.
- [3] J. Kabzan et al., “AMZ Driverless: The Full Autonomous Racing System,” *Journal of Field Robotics*, vol. 37, no. 7, pp. 1267–1294, 2020.
- [4] A. Betz et al., “Autonomous Vehicles on the Edge: A Survey on Autonomous Vehicle Racing,” *IEEE Open Journal of Intelligent Transportation Systems*, vol. 3, pp. 458–488, 2022.

13.2 Real-Time Iteration (RTI) and Embedded NMPC

- [5] M. Diehl, H. G. Bock, and J. P. Schlöder, “A Real-Time Iteration Scheme for Nonlinear Optimization in Optimal Feedback Control,” *SIAM Journal on Control and Optimization*, vol. 43, no. 5, pp. 1714–1736, 2005.
- [6] R. Verschueren, G. Frison, D. Kouzoupis, J. Frey, N. van Duijkeren, A. Zanelli, B. Nober, T. Albin, R. Quirynen, and M. Diehl, “acados—a modular open-source framework for fast embedded optimal control,” *Mathematical Programming Computation*, vol. 14, pp. 147–183, 2022.

13.3 ADMM and QP Solvers for Embedded Systems

- [7] B. Stellato, G. Banjac, P. Goulart, A. Bartlett, and S. Boyd, “OSQP: An Operator Splitting Solver for Quadratic Programs,” *Mathematical Programming Computation*, vol. 12, pp. 637–672, 2020.
- [8] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, “Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers,” *Foundations and Trends in Machine Learning*, vol. 3, no. 1, pp. 1–122, 2011.

13.4 FPGA-Accelerated MPC

- [9] J. L. Jerez, P. J. Goulart, S. Richter, G. A. Constantinides, E. C. Kerrigan, and M. Morari, “Embedded Online Optimization for Model Predictive Control at Megahertz Rates,” *IEEE Transactions on Automatic Control*, vol. 59, no. 12, pp. 3238–3251, 2014.
- [10] E. N. Hartley, J. L. Jerez, A. Suardi, J. M. Maciejowski, E. C. Kerrigan, and G. A. Constantinides, “Predictive Control Using an FPGA With Application to Aircraft Control,” *IEEE Transactions on Control Systems Technology*, vol. 22, no. 3, pp. 1006–1017, 2014.

- [11] J. L. Jerez, G. A. Constantinides, and E. C. Kerrigan, “An FPGA Implementation of a Sparse Quadratic Programming Solver for Constrained Predictive Control,” in *Proc. ACM/SIGDA International Symposium on FPGAs*, pp. 209–218, 2011.
- [12] G. Beléman, A. I. Reis, and S. S. Mukherjee, “FPGA-Based Model Predictive Control for Autonomous Driving,” in *Proc. IEEE International Conference on Field-Programmable Technology (FPT)*, 2020.

13.5 Vehicle Dynamics and Tire Modeling

- [13] R. Rajamani, *Vehicle Dynamics and Control*, 2nd ed. Springer, 2012.
- [14] H. B. Pacejka, *Tire and Vehicle Dynamics*, 3rd ed. Butterworth-Heinemann, 2012.

13.6 F1TENTH Platform

- [15] M. O’Kelly, H. Zheng, D. Karthik, and R. Mangharam, “F1TENTH: An Open-source Evaluation Environment for Continuous Control and Reinforcement Learning,” in *Proc. NeurIPS 2019 Competition and Demonstration Track (PMLR)*, vol. 123, pp. 77–89, 2020.
- [16] F1TENTH community resources: <https://f1tenth.org/>

13.7 HLS and Xilinx Tools

- [17] Xilinx/AMD, “Vitis High-Level Synthesis User Guide (UG1399),” 2025.
<https://docs.amd.com/r/en-US/ug1399-vitis-hls>
- [18] Xilinx/AMD, “Zynq UltraScale+ MPSoC Technical Reference Manual (UG1085),” 2025.
<https://docs.amd.com/r/en-US/ug1085-zynq-ultrascale-trm>